
FONDAMENTAUX DOCKER

INTRODUCTION ET INSTALLATION

1.1 Histoire de la conteneurisation:

Évolution des déploiements applicatifs

- **Années 1960-2000 : Applications déployées directement sur serveurs physiques**
 - Problème : "Ça marche sur ma machine"
 - Conflits de dépendances
 - Gaspillage de ressources
- **Années 2000-2010 : Virtualisation avec VMware, VirtualBox**
 - Isolation complète
 - Overhead important (OS complet par VM)
 - Démarrage lent (minutes)
- **2013 : Naissance de Docker**
 - Conteneurisation légère
 - Partage du noyau de l'OS hôte
 - Démarrage rapide (secondes)

Chronologie Docker

2013 : Docker Inc. lance Docker

2014 : Docker 1.0 - Production ready

2015 : Docker Compose, Docker Machine

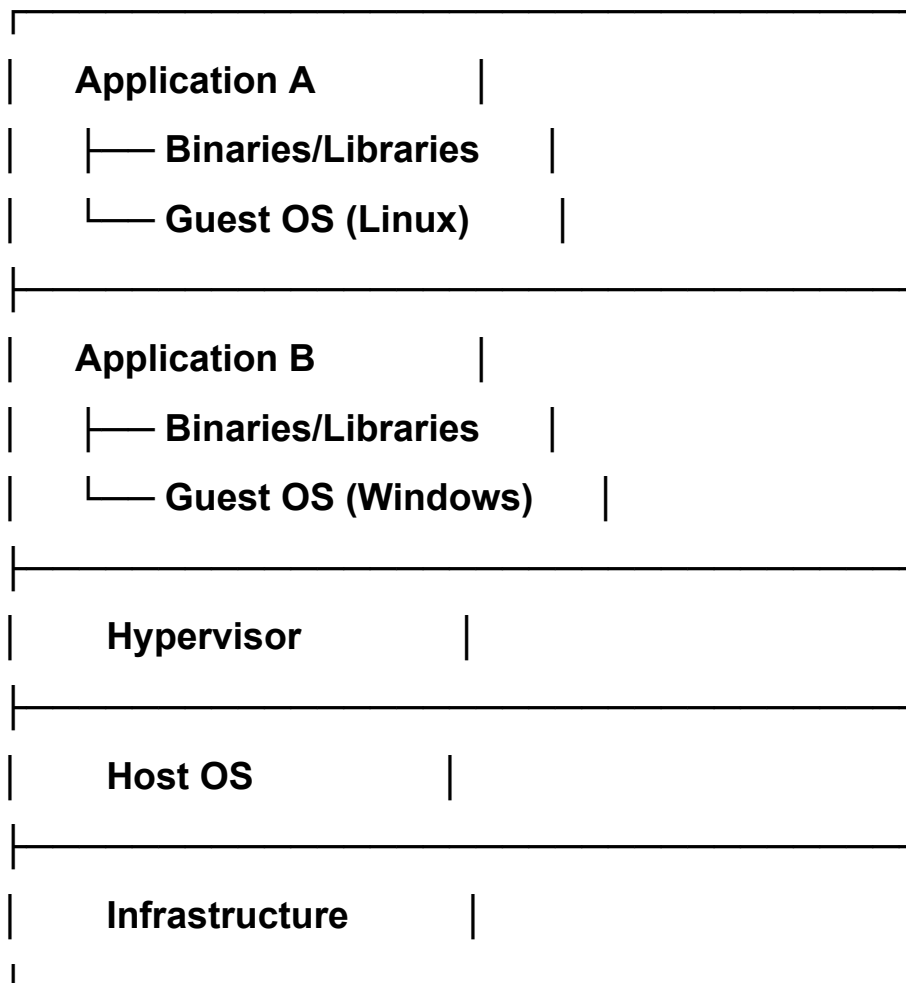
2016 : Docker Swarm intégré

2017 : Docker CE (Community) et EE (Enterprise)

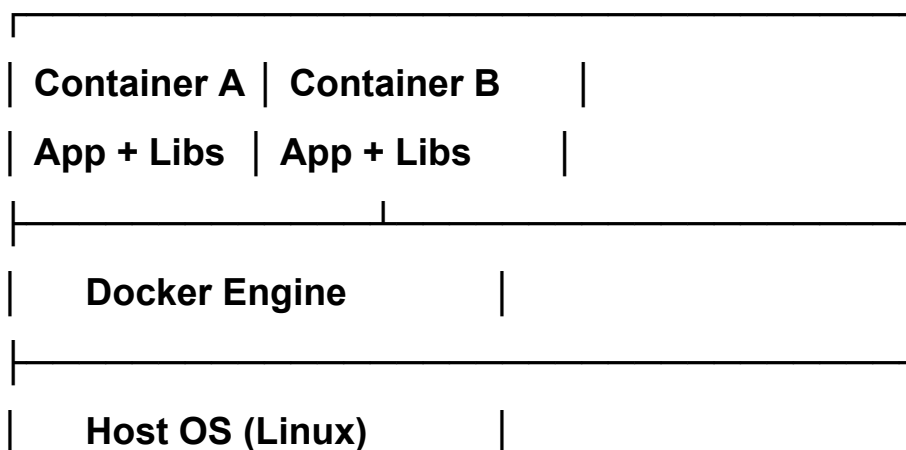
2020 : Docker Desktop nécessite licence pour grandes entreprises

1.2 Machines Virtuelles vs Conteneurs (45min)

Architecture VM



Architecture Conteneurs



Infrastructure	

Comparaison détaillée

Aspect	VM	Conteneur
Unité	(OS complet)	(application + dépendances)
Démarrage	lentes	rapides
Performance	Overhead de virtualisation	Proxime (quasi)
Isolation	Complète (hardware virtuel)	Processus (namespaces)
Portabilité	Lourde (images volumineuses)	Facile
Densité	~20 VMs par serveur	100+ conteneurs

1.3 Architecture Docker (45min)

Les composants de Docker

1. Docker Daemon (dockerd)

- Service backend qui tourne en arrière-plan
- Gère les images, conteneurs, networks, volumes
- Écoute les requêtes de l'API Docker
- Localisation : **/var/lib/docker/** (Linux)

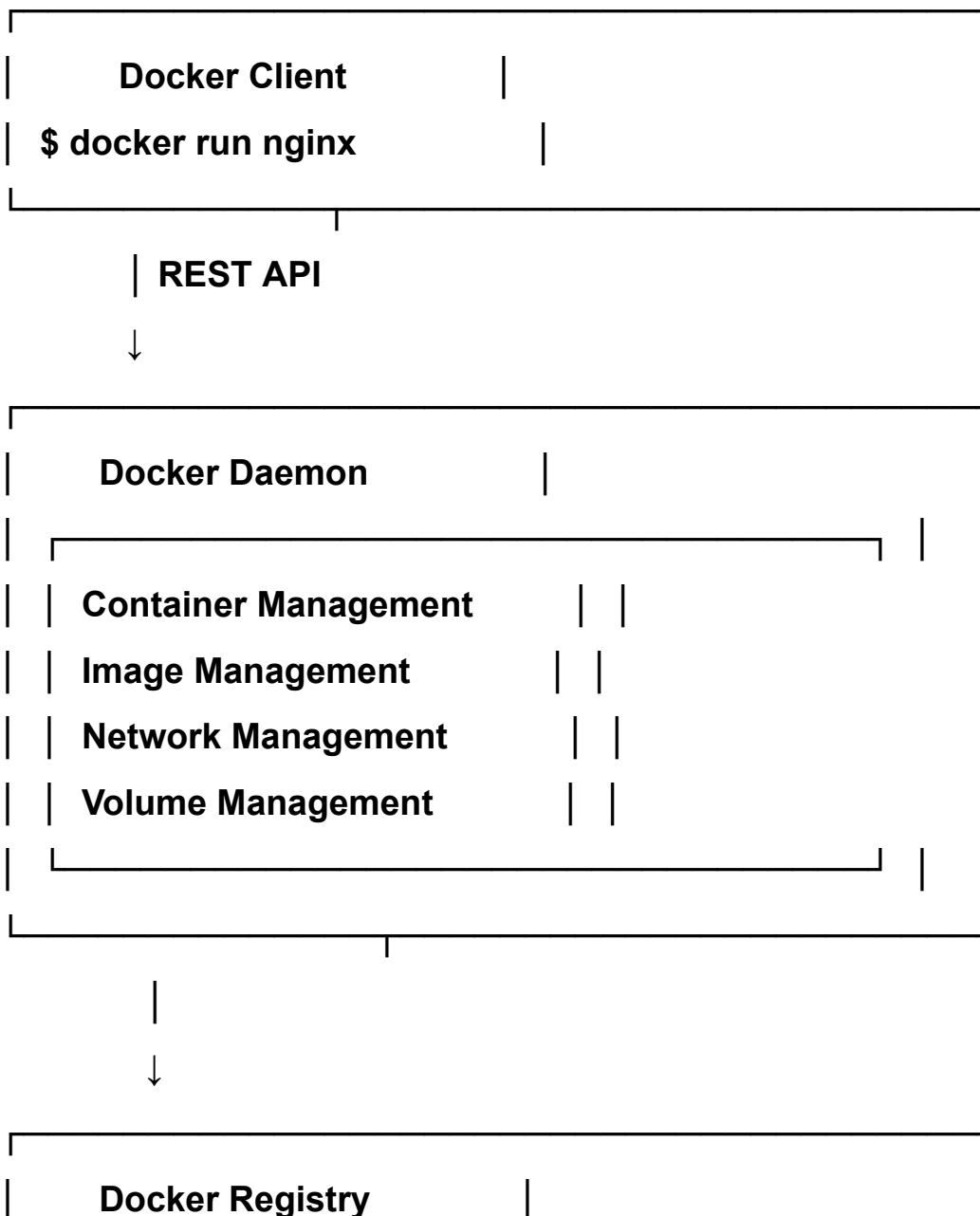
2. Docker Client (docker)

- Interface en ligne de commande
- Envoie les commandes au daemon via REST API
- Peut communiquer avec des daemons distants

3. Docker Registry

- Stocke les images Docker
- Docker Hub : registry public par défaut
- Registries privés possibles

Architecture détaillée



| (Docker Hub) |

1.4 Concepts clés Docker

Image Docker

- Modèle en lecture seule
- Contient le système de fichiers et les paramètres d'exécution
- Construite en couches (layers)
- Immuable une fois créée

Conteneur Docker

- Instance exécutable d'une image
- Ajoute une couche en lecture/écriture au-dessus de l'image
- Éphémère par défaut
- Isolé des autres conteneurs

Volumes

- Stockage persistant
- Survit à la suppression du conteneur
- Partageable entre conteneurs

Networks

- Isolation réseau entre conteneurs
- Communication inter-conteneurs
- Exposition de ports vers l'hôte

Registry

- Dépôt d'images
- Public (Docker Hub) ou privé
- Gestion des versions via tags

MATIN - SESSION 2 (2h) : Installation et premiers pas

2.1 Installation selon l'OS (45min)

Linux (Ubuntu/Debian)

Mise à jour des paquets

```
sudo apt-get update
```

Installation des dépendances

```
sudo apt-get install -y \
```

```
ca-certificates \
```

```
curl \
```

```
gnupg \
```

```
lsb-release
```

Ajout de la clé GPG officielle

```
sudo mkdir -p /etc/apt/keyrings
```

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
```

```
sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

Configuration du repository

```
echo \
```

```
"deb [arch=$(dpkg --print-architecture)  
signed-by=/etc/apt/keyrings/docker.gpg] \
```

```
https://download.docker.com/linux/ubuntu \
```

```
$(lsb_release -cs) stable" | \
```

```
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Installation Docker

```
sudo apt-get update
```

```
sudo apt-get install -y docker-ce docker-ce-cli containerd.io  
docker-compose-plugin
```

Ajouter votre utilisateur au groupe docker

sudo usermod -aG docker \$USER

Redémarrer la session ou exécuter

newgrp docker

macOS

Via Homebrew

brew install --cask docker

Ou télécharger Docker Desktop depuis docker.com

Installer et lancer l'application

Windows

Activer WSL2 (Windows Subsystem for Linux)

wsl --install

Télécharger Docker Desktop depuis docker.com

Installer et configurer pour utiliser WSL2 backend

2.2 Vérification de l'installation (30min)

Commandes de diagnostic

Version de Docker

docker --version

Output: Docker version 24.0.6, build ed223bc

Informations détaillées

docker info

Analyse de docker info

docker info

Sortie importante à comprendre :

Client:

Context: default

Debug Mode: false

Server:

Containers: 5

Running: 2

Paused: 0

Stopped: 3

Images: 12

Server Version: 24.0.6

Storage Driver: overlay2

Backing Filesystem: extfs

Logging Driver: json-file

Cgroup Driver: cgroupfs

Plugins:

Volume: local

Network: bridge host ipvlan macvlan null overlay

CPUs: 8

Total Memory: 15.61GiB

Docker Root Dir: /var/lib/docker

Comprendre les éléments

- **Storage Driver : Mécanisme de stockage des layers (overlay2 est le plus performant)**
- **Cgroup Driver : Gestion des ressources**
- **Docker Root Dir : Où Docker stocke tout**
- **Plugins : Drivers disponibles pour volumes et networks**

2.3 Première commande : Hello World (45min)

La commande magique

docker run hello-world

Ce qui se passe en détail

- 1. Docker client contacte le Docker daemon**
- 2. Daemon cherche l'image 'hello-world' localement**
- 3. Ne la trouve pas, donc télécharge depuis Docker Hub**
- 4. Crée un nouveau conteneur depuis cette image**
- 5. Exécute l'application dans le conteneur**
- 6. Affiche le message**
- 7. Le conteneur s'arrête**

Sortie expliquée

Unable to find image 'hello-world:latest' locally

Docker ne trouve pas l'image en local

latest: Pulling from library/hello-world

Téléchargement depuis Docker Hub

2db29710123e: Pull complete

Layer téléchargé avec succès

Digest: sha256:80f31da1ac7b312ba29d65080...

Hash de vérification de l'image

Status: Downloaded newer image for hello-world:latest

Image téléchargée et prête

Hello from Docker!

Message de l'application

APRÈS-MIDI - SESSION 1 (2h) : Docker Hub et gestion d'images

3.1 Explorer Docker Hub (45min)

Qu'est-ce que Docker Hub ?

- **Registry public le plus populaire**
- **Images officielles maintenues**
- **Images communautaires**
- **Gratuit pour images publiques**
- **Plans payants pour privé**

Navigation sur hub.docker.com

Images officielles à connaître

Systèmes d'exploitation

ubuntu

alpine **# Linux ultra-léger (5MB)**

debian

centos

Langages de programmation

python

node

golang

openjdk

ruby

Bases de données

postgres

mysql

mongodb

redis

elasticsearch

Serveurs web

nginx

apache

caddy

Outils

jenkins

gitlab-ce

wordpress

Comprendre les tags

Format : nom_image:tag

nginx:latest # Dernière version

nginx:1.25 # Version spécifique

nginx:1.25-alpine # Version + distribution

python:3.11-slim # Version optimisée

node:18-alpine # Petit + spécifique

3.2 Télécharger des images (45min)

Commande docker pull

Syntaxe de base

docker pull [OPTIONS] IMAGE[:TAG]

Exemples

docker pull ubuntu

docker pull ubuntu:22.04

docker pull ubuntu:latest

docker pull nginx:alpine

Exercice pratique : Télécharger plusieurs images

Images de base

docker pull alpine

docker pull ubuntu:22.04

docker pull debian:bullseye-slim

Serveurs web

docker pull nginx:alpine

docker pull httpd:alpine

Langages

docker pull python:3.11-slim

docker pull node:18-alpine

Base de données

docker pull postgres:15-alpine

docker pull redis:alpine

Vérifier les images téléchargées

docker images

ou

docker image ls

Sortie

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
nginx	alpine	0e0f89e0dfab	2 weeks ago	23.5MB
ubuntu	22.04	3b418d7b466a	3 weeks ago	77.8MB
python	3.11-slim	4b142c7b2d5a	1 month ago	125MB
postgres	15-alpine	9f71a9f62f7e	2 months ago	238MB

Comprendre la sortie

- **REPOSITORY** : Nom de l'image
- **TAG** : Version/variante
- **IMAGE ID** : Identifiant unique (SHA256 tronqué)
- **CREATED** : Date de création
- **SIZE** : Taille compressée

3.3 Gestion des images (30min)

Inspecter une image

`docker image inspect nginx:alpine`

Sortie JSON avec informations importantes

```
{  
  "Id": "sha256:0e0f89e0dfab...",  
  "RepoTags": ["nginx:alpine"],  
  "Created": "2024-01-15T10:20:30.123456789Z",  
  "Architecture": "amd64",  
  "Os": "linux",  
  "Size": 23456789,  
  "Layers": [  
    "sha256:abcd1234...",  
    "sha256:efgh5678..."  
  ],  
  "Env": [  
    "PATH=/usr/local/sbin:/usr/local/bin:...",  
    "NGINX_VERSION=1.25.3"  
  ],  
  "Cmd": ["nginx", "-g", "daemon off;"]  
}
```

Historique des layers

docker history nginx:alpine

Supprimer des images

Supprimer une image spécifique

docker rmi nginx:alpine

Supprimer plusieurs images

docker rmi ubuntu:22.04 debian:bullseye-slim

Supprimer toutes les images non utilisées

docker image prune

Supprimer toutes les images

docker image prune -a

APRÈS-MIDI - SESSION 2 (1h30) : Premiers conteneurs

4.1 Lancer un conteneur interactif (45min)

Commande docker run - Anatomie complète

docker run [OPTIONS] IMAGE [COMMAND] [ARG...]

Premier conteneur interactif

docker run -it ubuntu bash

Explication des options

- **-i** ou **--interactive** : Garde STDIN ouvert
- **-t** ou **--tty** : Alloue un pseudo-terminal
- **ubuntu** : Image à utiliser
- **bash** : Commande à exécuter

À l'intérieur du conteneur

Vous êtes maintenant dans le conteneur

```
root@a1b2c3d4e5f6:/#
```

Explorer le système

```
ls /
```

```
cat /etc/os-release
```

```
whoami      # root
```

```
hostname    # ID du conteneur
```

```
ps aux      # Processus en cours
```

Exercices dans le conteneur

1. Mettre à jour les packages

```
apt-get update
```

2. Installer des outils

```
apt-get install -y curl wget vim
```

3. Créer des fichiers

```
echo "Hello from container" > /tmp/test.txt
```

```
cat /tmp/test.txt
```


4. Installer Python

```
apt-get install -y python3 python3-pip
```

5. Créer un script Python

```
cat > /tmp/hello.py << EOF
```

```
#!/usr/bin/env python3
```

```
print("Hello from Python in Docker!")
```

```
EOF
```

```
python3 /tmp/hello.py
```

6. Quitter le conteneur

```
exit
```

4.2 Comprendre l'éphémérité (45min)

Relancer le même conteneur ?

Lancer un nouveau conteneur Ubuntu

```
docker run -it ubuntu bash
```

À l'intérieur, vérifier

```
ls /tmp/
```

Le fichier test.txt n'existe pas !

C'est un NOUVEAU conteneur

Concept clé : Éphémérité

- Chaque **docker run** crée un NOUVEAU conteneur
- Les modifications sont perdues à la sortie
- Les conteneurs sont immuables par design

Voir tous les conteneurs

Conteneurs en cours d'exécution

docker ps

Tous les conteneurs (incluant arrêtés)

docker ps -a

Sortie de docker ps -a

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
a1b2c3d4e5f6	ubuntu	"bash"	5 minutes ago	Exited (0) 2 minutes ago
f6e5d4c3b2a1	ubuntu	"bash"	10 minutes ago	Exited (0) 8 minutes ago

Redémarrer un conteneur existant

Démarrer un conteneur arrêté

docker start a1b2c3d4e5f6

Attacher au conteneur (mode interactif)

docker attach a1b2c3d4e5f6

Alternative : exec dans un conteneur en cours

docker exec -it a1b2c3d4e5f6 bash

Différence start vs run

- **docker run** : Crée un NOUVEAU conteneur
 - **docker start** : Redémarre un conteneur EXISTANT
-

APRÈS-MIDI - SESSION 3 (1h30) : Gestion des conteneurs

5.1 Cycle de vie d'un conteneur (30min)

États d'un conteneur

Created → Running → Paused → Stopped → Removed



Commandes de gestion

Créer sans démarrer

```
docker create --name mon-conteneur ubuntu
```

Démarrer

```
docker start mon-conteneur
```

Arrêter (SIGTERM, puis SIGKILL après 10s)

```
docker stop mon-conteneur
```

Arrêt immédiat (SIGKILL)

```
docker kill mon-conteneur
```

Redémarrer

```
docker restart mon-conteneur
```

Pause (freeze les processus)

docker pause mon-conteneur

Reprendre

docker unpause mon-conteneur

Supprimer (doit être arrêté)

docker rm mon-conteneur

Forcer la suppression (même en cours)

docker rm -f mon-conteneur

5.2 Options importantes de docker run (30min)

Nommer un conteneur

docker run --name mon-nginx nginx

Plus facile que d'utiliser l'ID

Mode détaché (background)

docker run -d nginx

Rend la main immédiatement

Conteneur tourne en arrière-plan

Exposer des ports

Syntaxe : -p HOST_PORT:CONTAINER_PORT

docker run -d -p 8080:80 nginx

Tester

curl http://localhost:8080

Variables d'environnement

docker run -e MY_VAR=valeur -e ANOTHER_VAR=123 ubuntu env

Combiner les options

**docker run -d \
--name mon-site-web \
-p 8080:80 \
-e NGINX_HOST=example.com \
nginx:alpine**

5.3 Logs et inspection (30min)

Voir les logs

Logs d'un conteneur

docker logs mon-nginx

Suivre les logs en temps réel

docker logs -f mon-nginx

Dernières 100 lignes

docker logs --tail 100 mon-nginx

Logs avec timestamps

docker logs -t mon-nginx

Inspecter un conteneur

docker inspect mon-nginx

Sortie JSON importante

```
{  
  "Id": "a1b2c3...",  
  "State": {  
    "Status": "running",  
    "Running": true,  
    "Pid": 12345,  
    "StartedAt": "2024-01-15T10:30:00Z"  
  },  
  "NetworkSettings": {  
    "IPAddress": "172.17.0.2",  
    "Ports": {  
      "80/tcp": [{"HostPort": "8080"}]  
    }  
  },  
  "Mounts": []  
}
```

Statistiques en temps réel

docker stats mon-nginx

Sortie

CONTAINER	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O
mon-nginx	0.01%	2.5MiB / 15.61GiB	0.02%	648B / 0B	0B / 0B

MINI-PROJET DU JOUR : Application Python conteneurisée

Objectif

Créer un conteneur Ubuntu, y installer Python, créer une application simple et comprendre pourquoi les données sont perdues.

Étapes détaillées

1. Préparer l'environnement local

Créer un dossier de projet

mkdir ~/docker-jour1

cd ~/docker-jour1

2. Lancer un conteneur Ubuntu

docker run -it --name python-test ubuntu:22.04 bash

3. Dans le conteneur, installer Python

Mise à jour

apt-get update

Installation Python

apt-get install -y python3 python3-pip

Vérifier

python3 --version

4. Créer une application Python

Créer le fichier

cat > /app.py << 'EOF'

#!/usr/bin/env python3

def calculer_factorielle(n):

if n == 0 or n == 1:

return 1

return n * calculer_factorielle(n - 1)

def main():

print("=== Calculateur de Factorielle ===")

nombre = int(input("Entrez un nombre : "))

resultat = calculer_factorielle(nombre)

print(f"La factorielle de {nombre} est {resultat}")

if __name__ == "__main__":

main()

EOF

Rendre exécutable

chmod +x /app.py

5. Tester l'application

python3 /app.py

Entrez 5

Résultat : 120

6. Quitter le conteneur

exit

7. Essayer de redémarrer le conteneur

Le conteneur existe toujours

docker ps -a | grep python-test

Le redémarrer

docker start python-test

S'y attacher

docker attach python-test

OU exécuter une commande

docker exec -it python-test python3 /app.py

8. Constater que les fichiers sont là

docker exec python-test ls /

app.py est présent !

9. Supprimer et recréer

Supprimer le conteneur

```
docker rm -f python-test
```

Créer un nouveau conteneur

```
docker run -it --name python-test-2 ubuntu:22.04 bash
```

Vérifier

```
ls /
```

app.py n'existe pas !

C'est un NOUVEAU conteneur, vierge

Questions de réflexion

- 1. Pourquoi les données persistent dans un conteneur arrêté mais pas supprimé ?**
- 2. Comment pourrait-on sauvegarder notre application ?**
- 3. Quels sont les avantages de cette éphémérité ?**

Réponses

- 1. Le conteneur arrêté conserve sa couche d'écriture**
 - 2. Avec des volumes Docker ou en créant une nouvelle image**
 - 3. Environnements propres, reproductibles, pas de "pollution"**
-

RÉCAPITULATIF JOUR 1

Concepts maîtrisés ✓

- Architecture Docker (daemon, client, registry)**
- Différence VM vs Conteneurs**
- Docker Hub et images**
- Lancement de conteneurs interactifs**
- Cycle de vie des conteneurs**
- Éphémérité et immutabilité**
- Commandes essentielles : run, ps, logs, exec, rm**

Commandes apprises

docker --version
docker info
docker pull <image>
docker images
docker run -it <image> <command>
docker ps / docker ps -a
docker start/stop/restart <container>
docker exec -it <container> <command>
docker logs <container>
docker inspect <container>
docker rm <container>
docker rmi <image>

Pour demain

Nous allons apprendre à créer nos propres images Docker avec des Dockerfiles, pour ne plus perdre nos modifications !

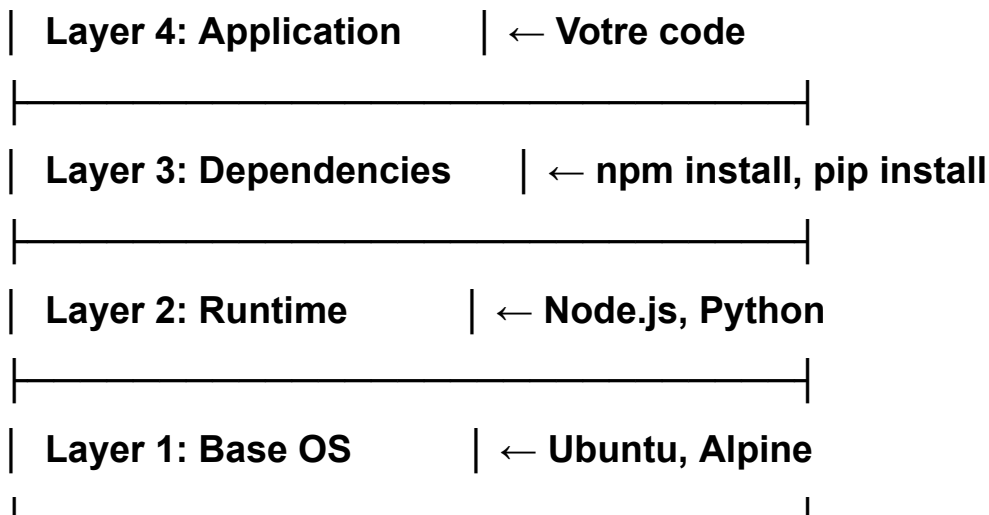
JOUR 2 : IMAGES DOCKER ET DOCKERFILE

MATIN - SESSION 1 (1h30) : Anatomie d'une image Docker

1.1 Qu'est-ce qu'une image Docker ? (30min)

Définition technique Une image Docker est un système de fichiers en couches (layers) empilées, avec des métadonnées d'exécution.

Structure en couches (Union File System)



Pourquoi des layers ?

Exemple concret

docker history nginx:alpine

Sortie

IMAGE	CREATED	CREATED BY	SIZE
0e0f89e0dfab	2 weeks ago	CMD ["nginx" "-g" "daemon off;"]	0B
<missing>	2 weeks ago	EXPOSE 80	0B
<missing>	2 weeks ago	COPY nginx.conf /etc/nginx/nginx.conf	2.5kB
<missing>	2 weeks ago	RUN apk add --no-cache nginx	9.2MB
<missing>	3 weeks ago	/bin/sh -c #(nop) ADD file:abc123... in /	7.3MB

Avantages des layers

1. Réutilisation : Layers partagés entre images
2. Efficacité : Téléchargement uniquement des layers modifiés

- 3. Cache : Accélération des builds**
- 4. Stockage : Économie d'espace disque**

Exemple de partage

Image A (100MB):	Image B (105MB):
— Ubuntu: 75MB	— Ubuntu: 75MB (partagé !)
— Python: 20MB	— Python: 20MB (partagé !)
└ App A: 5MB	└ App B: 10MB

Espace total utilisé: 110MB au lieu de 205MB

1.2 Images vs Conteneurs (45min)

Analogie Classe vs Instance

Image → Classe (modèle)

Conteneur → Instance (objet)

Image nginx → Modèle de serveur web

Conteneur 1 → Site web A

Conteneur 2 → Site web B

Conteneur 3 → Site web C

Caractéristiques des images

- **Immuables** : Une fois créées, ne changent jamais
- **Versionnées** : Via tags (v1.0, v2.0, latest)
- **Composables** : Se construisent les unes sur les autres
- **Distribuables** : Via registries (Docker Hub, etc.)

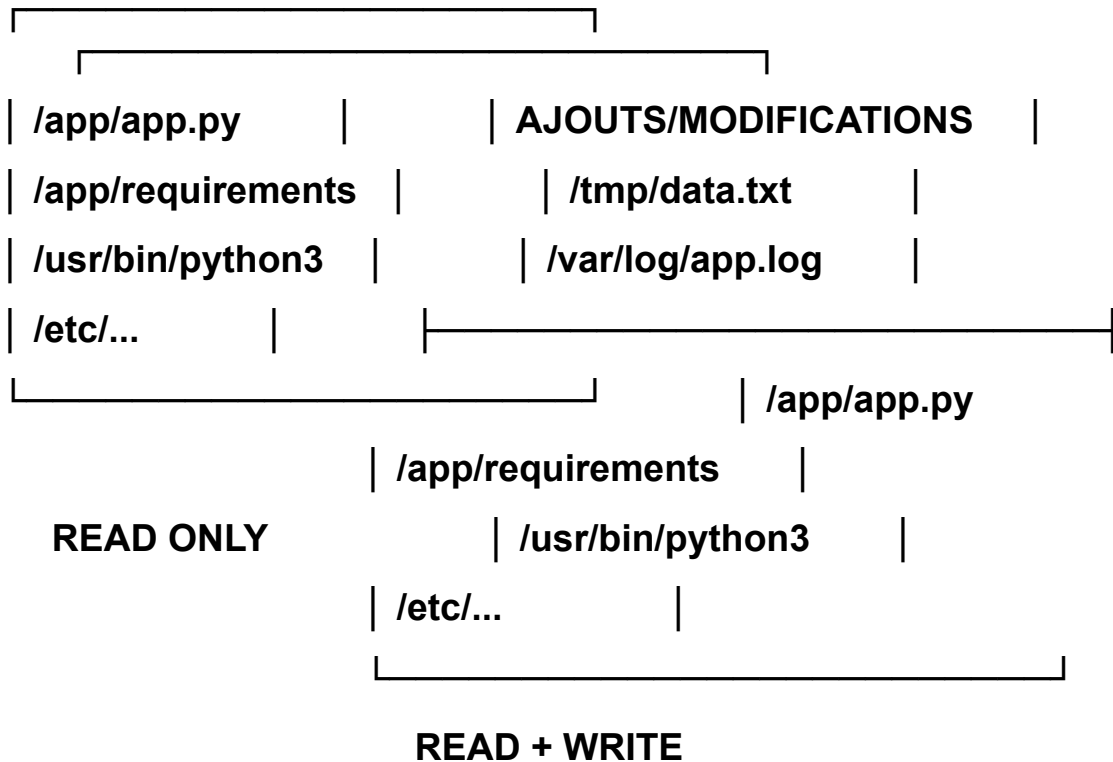
Caractéristiques des conteneurs

- **Éphémères** : Détruits et recréés facilement
- **Isolés** : Propres processus, réseau, filesystem

- **Légers** : Partage du kernel de l'hôte
- **Modifiables** : Couche d'écriture ajoutée au-dessus de l'image

Visualisation du système de fichiers

Image (lecture seule): Conteneur = Image + Couche RW:



1.3 Registries et tags (15min)

Docker Hub - Structure

[NAMESPACE]/[REPOSITORY]:[TAG]

Exemples:

nginx → Image officielle, latest implicite

nginx:1.25 → Version spécifique

nginx:alpine → Variante

username/myapp:v1.0 → Image perso avec version

username/myapp:latest → Image perso, dernière version

Convention de tags

Version sémantique

myapp:1.2.3

myapp:1.2

myapp:1

Environnement

myapp:prod

myapp:staging

myapp:dev

Distribution

myapp:alpine

myapp:debian

myapp:slim

Combinaisons

myapp:1.2.3-alpine

myapp:latest-slim

Bonnes pratiques de tagging

✗ À éviter

docker build -t myapp . # Pas de version

✓ Bon

```
docker build -t myapp:1.0.0 .  
docker build -t myapp:1.0 .  
docker build -t myapp:1 .  
docker build -t myapp:latest .
```

MATIN - SESSION 2 (2h30) : Dockerfile - Les bases

2.1 Structure d'un Dockerfile (30min)

Qu'est-ce qu'un Dockerfile ? Fichier texte contenant les instructions pour construire une image Docker de manière automatisée et reproductible.

Syntaxe de base

Commentaire

INSTRUCTION arguments

Premier Dockerfile simple

Spécifier l'image de base

FROM ubuntu:22.04

Mettre à jour les packages

RUN apt-get update && apt-get install -y python3

Définir le répertoire de travail

WORKDIR /app

#

